

KHUDA 10기 토이프로젝트 3조

이번 예측 모델 선정 보고서

어떤 모델을 써야 하는가 — 8종 비교 실험

머신러닝을 막 배운 사람도 읽을 수 있도록 용어부터 풀어 씀

항목	내용
작성일	2026년 8월 22일
대상	다크호스 예측 · 인기마 붕괴 예측
데이터	서울 경마 32,888행 · 시간순 6:2:2 분할
비교 모델	로지스틱 회귀 · 랜덤포레스트 · XGBoost · LightGBM (불균형 처리 유무 포함 8종)
결론	랜덤포레스트 권장. 불균형 처리는 하지 않음. 평가는 AUC 대신 Lift

0. 세 줄 요약

① 모델은 랜덤포레스트를 쓰세요

XGBoost·LightGBM보다 일관되게 좋았습니다. 표본이 작고 신호가 약한 문제라서, 잔차를 반복해 맞추는 부스팅보다 여러 나무의 평균을 내는 배깅이 유리합니다.

② 불균형 처리(class_weight, SMOTE)는 하지 마세요

여덟 번 실험했는데 전부 성능이 떨어지거나 그대로였습니다. 기저율 13.5%는 심한 불균형이 아니고, 우리는 순위로 평가하지 합격·불합격을 나누지 않기 때문입니다.

③ ROC-AUC로 판단하지 마세요

AUC 0.64는 낮아 보이지만 같은 모델의 Lift는 1.9배입니다. 상위 10%만 골랐을 때 적중률이 두 배가 된다는 뜻이고, 이쪽이 실제로 의미 있는 숫자입니다.

먼저 바로잡을 것 — 이변의 정의

데이터의 upset_B는 '비인기마의 1착'이 아니라 '비인기마의 입상(3착 이내)'입니다. 실제 데이터에서 역산해 확인했습니다.

upset_B = 1 인 행의 place(입상) 비율은 100%이고, win(1착) 비율은 19.8%뿐이었습니다.

이 차이가 중요한 이유는 수익 계산이 달라지기 때문입니다. 입상을 맞혔으면 단승이 아니라 연승 배당을 받습니다. 단승 배당으로 계산하면 수익률이 실제보다 몇 배 부풀려집니다.

1. 이변이란 무엇인가

경마를 모르는 분을 위해 정의부터 정리합니다.

1.1 인기마와 비인기마

배당률이 낮은 말이 인기마입니다. 사람들이 돈을 많이 걸었다는 뜻이니깐요. 우리 데이터에는 이걸 백분위로 만든 pop_pct 라는 값이 있습니다.

pop_pct	의미	평균 배당	실제 승률
0.0 ~ 0.2	가장 인기 있는 말	3.8배	26.6%
0.2 ~ 0.4		9.7배	10.2%
0.4 ~ 0.6	중간	19.2배	5.2%
0.6 ~ 0.8		37.2배	2.6%
0.8 ~ 1.0	가장 인기 없는 말	66.1배	1.2%

값이 클수록 비인기마입니다. 그리고 시장이 꽤 정확합니다 — 인기 순서대로 실제 승률이 26.6% → 1.2%로 깔끔하게 내려갑니다.

1.2 이변은 두 종류입니다

데이터에서 정의를 역산한 결과입니다. 일치율까지 확인했습니다.

이름	열	정의	쉬운 말로
다크호스	upset_B	인기 하위 50% 이면서 입상(3착 이내)	무시당하던 말이 잘 났을 경우
인기마 붕괴	upset_A	인기 상위 25% 이면서 착순 하위 50%	믿었던 말이 무너진 경우

정의 검증

upset_A = (pop_pct ≤ 0.25) & (fin_pct ≥ 0.50) → 일치율 100.0%

upset_B = (pop_pct ≥ 0.50) & (place = 1) → 입상 비율 100%, 1착 비율 19.8%

둘은 서로 반대 현상입니다. 그래서 합쳐서 하나의 모델로 학습하면 신호가 상쇄됩니다. 반드시 나눠서 각각 모델을 만들어야 합니다.

1.3 왜 이걸 예측하려 하는가

우리 프로젝트의 원래 목표는 '1착 맞추기'였는데 시장(배당률)을 넘지 못했습니다. 시장 ROC-AUC 0.81 대 모델 0.77입니다.

그래서 목표를 바꿔봤습니다. 시장 전체와 정면으로 겨루는 대신, **시장이 틀리는 지점만** 노리는 겁니다. 시장이 무시한 말이 잘 뛰는 경우를 찾아내면, 배당이 높으니 적중 시 보상도 큼니다.

2. 이견 보통 예측 문제와 다릅니다

모델을 고르기 전에 문제의 성격을 봐야 합니다. 세 가지가 특이합니다.

2.1 부분집합에서만 정의됩니다

다크호스는 '비인기마'에게만 물을 수 있는 질문입니다. 1인기마에게 '너는 다크호스니?'라고 묻는 건 말이 안 됩니다. 그래서 학습 데이터가 절반으로 줄어듭니다.

	전체	다크호스 (인기 하위 50%)	인기마 붕괴 (인기 상위 25%)
train	19,637	10,374	5,597
valid	6,591	3,490	1,881
test	6,660	3,528	1,902
기저율 (test)	9.56% (1착)	13.55%	25.45%

인기마 붕괴는 학습 데이터가 **5,597행**밖에 안 됩니다. 원래의 3분의 1도 안 되는 규모입니다. 이게 모델 선택에 결정적인 영향을 줍니다.

2.2 신호가 약합니다

시장이 이미 대부분의 정보를 반영했기 때문에, 남아 있는 신호가 작습니다. 뒤에서 보겠지만 어떤 모델을 써도 ROC-AUC가 0.64를 넘지 못합니다.

이게 무슨 뜻이냐면 — **모델이 조금만 복잡해져도 신호 대신 잡음을 외우기 시작한다**는 뜻입니다. 정규화를 세게 걸어야 합니다.

2.3 순위만 맞으면 됩니다

우리는 '이 말이 이벤을 일으킬 확률이 정확히 몇 %인가'를 알 필요가 없습니다. **상위 몇 마리만 골라내면 되는** 문제입니다.

이 차이가 평가 지표와 불균형 처리 결정을 바꿉니다. 8장에서 다시 다룹니다.

3. 후보 모델 — 무엇을 왜 넣었나

네 종류를 골랐습니다. 각각 다른 가정을 하고 있어서, 어느 쪽이 이 문제에 맞는지 알아보려는 목적입니다.

3.1 로지스틱 회귀 — 가장 단순한 기준선

모든 변수에 가중치를 하나씩 곱해서 더하는 방식입니다. 직선 하나로 경계를 긋는다고 생각하면 됩니다.

$$\log(p / (1-p)) = b_0 + b_1 \cdot x_1 + b_2 \cdot x_2 + \dots + b_k \cdot x_k$$

왼쪽은 로그 오즈. 확률 p 를 0~1 밖으로 늘려서 직선으로 다룰 수 있게 만든 것

숫자로 보면 이렇습니다. $b_0 + b_1 x_1 + \dots$ 를 계산한 값이 -1.86이라면, 오즈는 $\exp(-1.86) = 0.156$, 확률은 $0.156 / (1 + 0.156) = 13.5\%$ 입니다. 우리 다크호스 기저율과 같습니다.

- **장점** — 변수가 많아도 안정적입니다. 표본이 적을 때 특히 강합니다. 계수를 보면 어떤 변수가 어느 방향으로 작용하는지 바로 읽힙니다.
- **단점** — 직선만 그을 수 있습니다. '휴식일이 20일까지는 좋다가 그 이상은 나쁘다' 같은 꺾인 관계를 못 잡습니다.
- **설정** — $C = 0.1$ (정규화 강도). 값이 작을수록 계수를 0 쪽으로 강하게 누릅니다. 표준화(z-score) 필수.

3.2 랜덤포레스트 — 나무 여러 개의 평균

결정나무는 '이 조건이면 왼쪽, 아니면 오른쪽'을 반복하는 스무고개입니다. 하나만 쓰면 데이터를 통째로 외워버리는데 (과적합), 랜덤포레스트는 이 문제를 두 겹의 무작위성으로 해결합니다.

1. 데이터를 무작위 복원추출해서 나무마다 다른 표본을 줍니다 (배깅).
2. 각 분기점에서 전체 변수 중 일부만 후보로 씁니다.
3. 이렇게 만든 600그루의 예측 확률을 평균냅니다.

핵심은 나무들이 서로 다른 실수를 한다는 점입니다. 실수 방향이 제각각이면 평균낼 때 상쇄됩니다. 신호는 모든 나무가 공통으로 잡으니 남습니다.

왜 이게 우리 문제에 맞는가

신호가 약할 때 개별 나무의 예측은 거의 잡음입니다. 그런데 잡음은 평균내면 줄어들고 신호는 남습니다.

부스팅(XGBoost·LightGBM)은 반대입니다. 앞 나무가 틀린 부분을 다음 나무가 집중해서 맞춥니다. 신호가 강하면 강력하지만, 잡음이 많으면 잡음을 열심히 외웁니다.

- **장점** — 비선형·상호작용을 자동으로 잡습니다. 하이퍼파라미터에 둔감해서 대충 맞춰도 잘 됩니다. 결측치와 스케일에 강합니다.

- **단점** — 느립니다(600그루). 확률값 자체는 극단으로 잘 안 갑니다 — 순위만 볼 거라 문제되지 않습니다.
- **설정** — `n_estimators=600`, `max_depth`는 검증셋으로 선택, `min_samples_leaf=50` (앞에 최소 50개 — 과적합 방지의 핵심)

3.3 XGBoost / LightGBM — 순차적으로 오답을 고치는 방식

나무를 하나씩 순서대로 만듭니다. n 번째 나무는 $(1 \sim n-1)$ 번 나무들이 놓친 부분을 맞추도록 학습됩니다. 오답노트를 반복해서 푸는 것과 같습니다.

보통의 표 형태 데이터에서는 이 방식이 최강입니다. Kaggle 대회 상위권이 대부분 이걸 씁니다. 그래서 당연히 후보에 넣었습니다.

그런데 조건이 있습니다

오답노트 전략은 오답에 진짜 배울 게 있을 때 통합니다.

우리 문제에서 놓친 케이스 상당수는 그냥 운입니다. 배우려 하면 잡음을 외우게 됩니다.

게다가 학습 데이터가 5,597 ~ 10,374행뿐이라, 400그루를 순차적으로 쌓으면 금방 데이터를 소진합니다.

그래서 정규화를 최대로 걸었습니다. `max_depth=3` (매우 얇게), `reg_lambda=10` (L2 페널티), `subsample=0.8`, `colsample_bytree=0.8`. 이렇게까지 했는데도 결과는 뒤에서 보시는 대로입니다.

3.4 무작위 기준선

난수를 예측값으로 쓴 모델입니다. 모든 지표가 '아무 정보도 없을 때' 어떤 값이 나오는지 알려줍니다. ROC-AUC 0.5, Lift 1.0 근처가 나와야 정상이고, 실제로 그랬습니다.

이게 없으면 '0.62면 좋은 건가?'를 판단할 수 없습니다.

4. 실험을 어떻게 설계했나

4.1 시간순 분할 — 미래를 훑쳐보지 않기

데이터를 무작위로 섞어 나누면 안 됩니다. 2024년 경주로 학습해서 2023년 경주를 맞히는 꼴이 되기 때문입니다. 실전에서는 불가능한 조건이죠.

과거 60% -> train | 중간 20% -> valid | 최근 20% -> test

경주일 기준으로 자릅니다. 같은 경주가 두 집합에 걸치지 않도록 `race_id` 단위로 나눕니다

valid는 하이퍼파라미터를 고를 때만 쓰고, test는 최종 보고 한 번만 봅니다. test를 보고 설정을 바꾸면 그 숫자는 더 이상 믿을 수 없게 됩니다.

4.2 배당률과 결과 정보를 전부 뺐습니다

타겟이 인기도로 정의되어 있어서, 배당률 관련 열이 하나라도 남으면 정답을 그대로 보는 셈이 됩니다. 제외한 것들입니다.

종류	제외한 열	이유
타겟 정의에 쓰인 것	pop_pct, upset_A, upset_B	정답 그 자체
시장 정보	winOdds, plcOdds, winAmt, plcAmt, totalAmt, log_winAmt, liq_per_horse	pop_pct와 사실상 같은 정보
경주 결과	ord, place, win, fold	경주가 끝나야 알 수 있음
식별자	race_id, entry_id, 이름 계열	학습에 의미 없음

남은 변수는 마필 과거 성적, 기수·조교사 통계, 부담중량, 휴식일, 트랙·날씨 등 경주 전에 알 수 있는 것들입니다.

검증 — 정보 누출이 있었다면?

만약 배당률이 새어 들어갔다면 ROC-AUC가 0.9를 넘었을 겁니다. 실제로는 0.64입니다.

반대로 실험 초기에 pop_pct를 빼먹었을 때는 AUC가 0.98이 나왔습니다. 그때 누출을 발견하고 제외 목록을 만들었습니다.

4.3 무엇을 비교했나

- 모델 4종 + 무작위 기준선
- 랜덤포레스트 깊이 5단계 (3 / 4 / 6 / 8 / 12)
- 불균형 처리 유무 (`class_weight='balanced'`, `scale_pos_weight`)

- 학습 범위 3가지 (부분집합만 / 전체 + 이변 라벨 / 전체 + 1차 라벨)
- 평가 지표 6종 (ROC-AUC, PR-AUC, Lift@5/10/20/30%)
- 수익성 (연승 배당 기준 ROI, 상위 k% 베팅)

5. 결과 — 다크호스 예측

test 3,528행, 기저율 13.55%. 이 중 478마리가 실제로 입상했습니다.

5.1 모델 비교

모델	ROC-AUC	PR-AUC	Lift@5%	Lift@10%	Lift@20%	Lift@30%
무작위	0.4881	0.1344	1.09	1.07	0.95	0.96
로지스틱 회귀	0.6195	0.1928	1.72	1.59	1.55	1.49
랜덤포레스트	0.6420	0.2093	2.01	1.72	1.61	1.45
XGBoost	0.6205	0.2058	1.89	1.70	1.49	1.43
LightGBM	0.6034	0.1803	1.34	1.38	1.35	1.33

랜덤포레스트가 6개 지표 중 5개에서 1위입니다. 특히 Lift@5%가 2.01 — 모델이 가장 확신하는 상위 5%(176마리)를 골랐더니 적중률이 27.3%로, 그냥 고를 때(13.55%)의 두 배였습니다.

숫자 읽는 법

Lift@10% = 2.01 의 의미

전체 3,528마리 중 상위 10% = 352마리를 골랐다

그 안에서 82마리가 입상 → 적중률 23.3%

아무나 골랐으면 13.55%

$23.3 / 13.55 = 1.72$ 배

Lift 1.0은 무작위와 같다는 뜻입니다. 1.72는 72% 더 잘 골랐다는 뜻이고요. 부트스트랩으로 신뢰구간을 잡으면 [1.41, 2.02] 로, 하한이 1을 넉넉히 넘습니다. 우연이 아닙니다.

5.2 왜 LightGBM이 꼴찌인가

LightGBM은 leaf-wise 방식입니다. 가장 이득이 큰 잎을 골라서 그쪽만 계속 쪼갭니다. 데이터가 많으면 효율적인데, 표본이 적으면 특정 소수 구간을 파고들다가 그 구간의 잡음까지 외웁니다.

XGBoost는 level-wise(깊이 단위로 고르게)라서 조금 낫지만, 여전히 순차 학습이라는 근본 성질은 같습니다. 그래서 랜덤포레스트와 로지스틱 사이에 애매하게 걸쳐 있습니다.

5.3 랜덤포레스트 깊이 조정

검증셋 ROC-AUC로 골랐습니다. min_samples_leaf=50이 이미 강한 제약이라, 깊이를 12까지 늘려도 크게 무너지지 않았습니다.

max_depth	특징	결과
3	매우 얕음. 사실상 몇 개 변수만 봄	과소적합 — 신호를 다 못 씀
4	안정적	test AUC 0.6409, Lift@10% 1.87
6	중간	무난
8	깊음	test AUC 0.6419, Lift@5% 2.01
12	매우 깊음 (leaf=50이 제동)	검증셋 최고 → 최종 선택. test AUC 0.6420

깊이 4든 12든 test 성능 차이는 0.001 수준입니다. 결정적인 건 깊이가 아니라 min_samples_leaf=50 쪽입니다. 앞마다 최소 50개를 요구하니, 나무가 아무리 깊어져도 한두 개 사례를 외우는 일이 구조적으로 막힙니다.

실무 조언

이 문제에서는 max_depth를 정밀하게 튜닝할 필요가 없습니다. 4~12 아무 값이나 쓰고, min_samples_leaf를 30~100 사이에서 조정하세요.

min_samples_leaf를 1(기본값)로 두면 성능이 확 떨어집니다. 이게 가장 중요한 설정입니다.

5.4 어떤 변수가 중요했나

순위	변수	무슨 뜻인가
1	train_runs_14__z	최근 14일 조교 횟수 (경주별 표준화). 훈련량이 많으면 컨디션이 올라온 것
2	hr_last_ord	직전 경주 착순. 최근에 잘 뛰었으면 다시 잘 뛸 확률이 높음
3	hr_last_poppct	직전 경주 인기도. 예전에 인기가 있었다가 지금 비인기면 저평가 신호
4	hr_rest_days__z	휴식일. 너무 짧으면 피로, 너무 길면 감각 저하
5	rating__z	마필 레이팅
6~10	hr_plcrate, tr_winrate__z, wg_diff__z, wgBudam__z, hr_style_sd	입상률 · 조교사 승률 · 체중 변화 · 부담중량 · 각질 변동성

상위 5개가 전부 마필의 최근 상태를 나타내는 변수입니다. 해석하면 이렇습니다 — 시장은 말의 통산 성적과 명성에는 잘 반응하지만, 최근 몇 주간의 조교량 변화와 컨디션 회복은 상대적으로 덜 반영합니다. 우리 모델이 잡아낸 게 바로 그 틈입니다.

3위의 `hr_last_poppct`가 특히 흥미롭습니다. '전에는 인기마였는데 지금은 아닌 말'을 골라내는 변수입니다. 시장이 최근 부진에 과잉 반응했을 가능성을 포착합니다.

6. 결과 — 인기마 붕괴 예측

test 1,902행, 기저율 25.45%. 인기 상위 25% 말 중 네 마리에 한 마리 꼴로 무너집니다.

6.1 모델 비교

모델	ROC-AUC	PR-AUC	Lift@5%	Lift@10%	Lift@20%	Lift@30%
무작위	0.4870	0.2495	0.95	0.85	0.95	0.92
로지스틱 회귀	0.5722	0.3134	1.53	1.47	1.34	1.21
랜덤포레스트	0.5768	0.3177	1.49	1.47	1.28	1.23
XGBoost	0.5783	0.3238	1.45	1.41	1.31	1.29
LightGBM	0.5610	0.3097	1.57	1.39	1.26	1.18

다크호스보다 훨씬 어렵습니다. 최고 AUC가 0.578로, 무작위(0.5)와의 거리가 다크호스의 절반 수준입니다.

Lift@10%도 1.47로, 다크호스의 1.72보다 낮습니다.

그리고 모델 간 차이가 거의 없습니다. 랜덤포레스트 0.5768, XGBoost 0.5783 — 이 정도 차이는 난수 시드를 바꾸면 뒤집힙니다.

6.2 왜 더 어려운가

- **표본이 적습니다** — 학습 데이터 5,597행. 다크호스의 절반입니다.
- **인기마는 정보가 많이 반영돼 있습니다** — 1인기마는 모두가 주목합니다. 부상 소문, 조교 상태, 기수 교체까지 배당에 이미 들어가 있습니다. 남은 틈이 작습니다.
- **붕괴는 대부분 우연입니다** — 출발 실패, 진로 방해, 그날 컨디션 — 데이터로 잡히지 않는 요인이 큼니다.

Lift@10% 부트스트랩 신뢰구간이 [1.21, 1.75] 로 하한이 1을 넘기는 합니다. 신호는 있습니다. 다만 약합니다.

6.3 여기서는 변수 성격이 다릅니다

순위	변수	무슨 뜻인가
1	age__pr	나이 백분위. 나이 든 인기마가 더 자주 무너짐
2	hr_rest_days__z	휴식일. 오래 쉰 뒤 복귀전이 위험
3	rating__z	레이팅
4	train_runs_14__z	최근 조교량. 적으면 준비 부족 신호

순위	변수	무슨 뜻인가
5~7	wg_diff_z, jk_winrate, hr_rest_days	체중 변화 · 기수 승률 · 휴식일(원본)

다크호스에서 1위였던 조교량이 여기서 4위이고, 대신 나이와 휴식일이 앞으로 나왔습니다. 방향도 반대입니다 —

다크호스는 '조교량이 늘어난 말'을 찾고, 인기마 붕괴는 '오래 쉬었고 나이가 있는데 조교량도 적은 말'을 찾습니다.

두 모델이 다른 것을 보고 있다는 증거입니다. 하나로 합치면 안 되는 이유이기도 합니다.

7. 하지 말아야 할 것 두 가지

실험하면서 '당연히 좋을 것 같은데 실제로는 나쁜' 선택을 두 개 발견했습니다. 둘 다 교과서에 흔히 나오는 처방입니다.

7.1 불균형 처리 — 전부 나빠졌습니다

양성 비율이 13.5%면 불균형 데이터입니다. 보통은 양성에 가중치를 주거나(class_weight='balanced'), 양성을 복제해서 늘립니다(SMOTE). 세 모델에 전부 적용해봤습니다.

모델	처리 없음	불균형 처리	변화
랜덤포레스트	0.6381	0.6264	-0.0117 악화
로지스틱 회귀	0.6195	0.6196	+0.0001 차이 없음
XGBoost	0.5293	0.4945	-0.0348 크게 악화

XGBoost는 무작위(0.5)보다도 나빠졌습니다. 이유를 풀어보면 이렇습니다.

1. 가중치를 주면 모델이 양성 쪽으로 예측을 밀어냅니다. 확률값 전체가 위로 이동합니다.
2. 그런데 우리는 '이 말이 양성이나 아니냐'를 판정하지 않습니다. 확률 순서대로 줄 세워서 위에서 몇 마리만 뽑습니다.
3. 전체를 똑같이 위로 밀면 순서는 안 바뀝니다. 지표가 좋아질 이유가 없습니다.
4. 대신 부작용은 생깁니다. 소수의 양성 사례에 과도한 비중이 실리면서, 그 사례들의 특이한 성질까지 학습합니다.

정리하면

불균형 처리는 임계값을 넘나 마냥 판정하는 문제에서 유용합니다. (예: 사기 탐지 — 차단할지 말지 결정해야 함)

순위만 쓰는 문제에서는 이득이 없고 손해만 있습니다. 우리는 후자입니다.

그리고 13.5%는 사실 심한 불균형도 아닙니다. 사기 탐지는 0.1% 수준입니다.

7.2 전체 데이터로 학습하기 — 신호가 사라집니다

'데이터가 적으니 전체 32,888행을 다 쓰면 낫지 않을까?' 당연한 생각이라 실험했습니다.

학습 방식	test ROC-AUC	판정
부분집합만 (인기 하위 50%) + 다크호스 라벨	0.6419	최고 — 채택
전체 데이터 + 1착 라벨	0.6303	쓸만하지만 조금 못함
전체 데이터 + 다크호스 라벨	0.4599	붕괴 — 무작위보다 나쁨

세 번째가 왜 무너지는지가 핵심입니다.

전체 데이터에서 다크호스 라벨은 대부분 0입니다. 인기마는 정의상 전부 0이니까요. 그러면 모델은 '이 말이 인기마인가?'만 학습하게 됩니다. 정작 우리가 알고 싶은 '비인기마 중 누가 뛰는가'는 배우지 못합니다.

모델이 배운 것: $P(\text{비인기마})$ <- 쓸모없음

우리가 원하는 것: $P(\text{잘 뛴다} | \text{비인기마})$

두 번째(전체 + 1착 라벨)가 0.6303으로 나쁘지 않은 것도 흥미롭습니다. 일반적인 승률 모델을 만든 다음 비인기마 구간에서만 순위를 보는 방식도 가능하다는 뜻입니다. 다만 부분집합 전용 모델이 더 낫습니다.

일반 원칙

부분집합에서만 정의되는 타겟은 그 부분집합에서 학습하세요.

이건 조건부 확률을 배우는 문제이고, 조건을 만족하는 데이터만 봐야 조건부 확률이 나옵니다.

8. 평가 지표 — AUC를 버리세요

모델을 고르려면 잣대가 필요한데, 이 문제에서는 흔히 쓰는 잣대가 잘못된 답을 줍니다.

8.1 ROC-AUC의 문제

ROC-AUC는 '양성 하나와 음성 하나를 뽑았을 때, 모델이 양성에 더 높은 점수를 줄 확률'입니다. 0.5가 무작위, 1.0이 완벽입니다.

우리 최고 모델이 0.6420입니다. 실패로 보입니다. 그런데 이 숫자는 전체 순서를 다 보는 지표라서, 하위권을 잘 정렬한 것도 점수에 들어갑니다.

실제로 우리가 쓰는 건 상위 5~20%뿐입니다. 400등과 3,000등의 순서가 맞는지 아무 상관이 없습니다.

8.2 Lift — 실제로 의미 있는 숫자

$$\text{Lift@k\%} = (\text{상위 k\% 안에서의 적중률}) / (\text{전체 적중률})$$

상위 10%를 골랐더니 적중률이 23.3%였고, 전체 적중률은 13.55%였습니다. 1.72배입니다. 실무에서 바로 쓸 수 있는 숫자입니다.

	ROC-AUC	Lift@10%	해석
무작위	0.4881	1.07	정보 없음
우리 모델	0.6420	1.72	72% 더 잘 고름

같은 모델인데 AUC는 '별로', Lift는 '괜찮음'이라고 말합니다. 둘 다 맞습니다. 전체 순서는 별로고, 상위권 선별은 괜찮습니다. 우리가 필요한 건 후자입니다.

8.3 PR-AUC도 같이 보세요

Precision-Recall AUC는 양성이 드물 때 ROC-AUC보다 민감합니다. 기저율이 그대로 기준선이 되는 것도 편합니다 — 다크호스는 0.1355가 무작위 수준이고, 우리 모델은 0.2093으로 1.55배입니다.

정리하면 이렇게 보세요.

- **Lift@k%** — 주 지표 : 실제 사용 방식과 일치합니다. k는 실제로 베팅할 비율로 잡으세요.
- **PR-AUC** — 보조 : 상위권 전반의 품질. Lift가 특정 k에서 우연히 좋은 건 아닌지 확인용.
- **ROC-AUC** — 참고만 : 다른 논문·프로젝트와 비교할 때만. 절대값으로 판단하지 마세요.

8.4 신뢰구간을 꼭 붙이세요

test 3,528행 중 양성인 478개뿐입니다. 상위 10%면 352마리에 적중 82건. 이 정도 표본에서 나온 숫자는 흔들립니다.

부트스트랩(데이터를 복원추출로 다시 뽑아 2,000번 반복)으로 구간을 잡았습니다.

	Lift@10%	95% 신뢰구간	판정
다크호스	1.72	[1.41, 2.02]	하한이 1을 넉넉히 넘음 — 신호 확실
인기마 붕괴	1.47	[1.21, 1.75]	하한이 1을 넘지만 여유가 적음

9. 수익성 — 실제로 돈이 되는가

모델이 잘 고른다는 건 확인했습니다. 그럼 실제로 걸면 돈이 될까요?

결론

표면 수치는 좋습니다. 상위 10% 베팅 ROI +65.3%.

그런데 이 수익의 전부가 260배 적중 1건에서 나옵니다. 그 한 건을 빼면 -8.4%입니다.

즉 현재 모델은 선별 능력은 입증됐지만, 수익성은 입증되지 않았습니다.

9.1 계산 방법

다크호스는 '입상'이므로 연승(plcOdds) 배당을 씁니다. 매 베팅 1,000원씩 균등하게 걸었다고 가정합니다.

$$\text{수익률} = (\text{적중 여부} \times \text{연승배당}) - 1$$

적중하면 배당만큼 받고 원금은 이미 나갔으므로 1을 뺍니다. 빚나가면 -1 (전액 손실)

예시 — 연승배당 4.2배 말에 1,000원

적중: $1,000 \times 4.2 - 1,000 = +3,200\text{원}$ (수익률 +320%)

실패: $0 - 1,000 = -1,000\text{원}$ (수익률 -100%)

10번 걸어서 2번 맞으면:

$$(2 \times 3,200 - 8 \times 1,000) / 10,000 = -0.16 \rightarrow \text{ROI } -16\%$$

왜 단승 배당을 쓰면 안 되는가

이 부분에서 실수가 있었습니다. 처음에는 upset_B를 '1착'으로 알고 단승(winOdds) 배당으로 계산했습니다.

그랬더니 무작위로 골라도 ROI +306%가 나왔습니다. 말이 안 되는 값이죠.

입상은 3착까지 인정되니 적중률이 3배 높은데, 배당은 훨씬 낮습니다. 연승 배당으로 바꿔 계산하니 무작위 ROI가 -22.6%로 정상화됐습니다. (파리뮤추얼 공제율 약 20%와 일치)

9.2 상위 k% 베팅 결과

모델 예측값 순으로 정렬해서 상위 k%에만 걸었을 때입니다. test 3,528마리 기준.

선택 범위	베팅 수	적중	적중률	평균 배당	ROI	95% 신뢰구간
상위 2%	70	22	31.4%	7.18	+364.1%	[-32.6%, +1154%]
상위 5%	176	48	27.3%	5.39	+144.0%	[-21.9%, +453%]

선택 범위	베팅 수	적중	적중률	평균 배당	ROI	95% 신뢰구간
상위 10%	352	82	23.3%	4.97	+65.3%	[-22.6%, +227%]
상위 20%	705	154	21.8%	4.89	+24.0%	[-23.8%, +104%]
상위 30%	1,058	208	19.7%	5.20	+6.2%	[-26.7%, +62%]
전체 균등	3,528	478	13.5%	8.04	-8.2%	[-30.3%, +20%]

적중률은 아주 깔끔하게 내려갑니다 — 31.4% → 27.3% → 23.3% → 21.8% → 19.7% → 13.5%. 모델 예측값이 실제 확률과 잘 맞아떨어진다는 뜻이고, 선별 능력의 강한 증거입니다.

ROI도 함께 내려가지만, 신뢰구간이 전부 0을 포함합니다. 상위 2%는 [-32.6%, +1154%] — 사실상 아무것도 말해주지 않는 구간입니다.

9.3 왜 신뢰구간이 이렇게 넓은가

고배당 적중 한 건이 결과를 지배하기 때문입니다. 상위 10% 선택의 적중마를 배당순으로 늘어놓으면 이렇습니다.

적중 82건의 연승배당 (상위 8개)

260.3배 <- 이 한 건

9.4배

8.8배

7.6배

7.6배

7.2배

6.5배

6.2배

... 나머지 74건은 대부분 2~5배

260.3배 적중 하나가 1,000원 베팅에서 259,300원을 벌어들입니다. 나머지 351번의 베팅 총액이 351,000원인데, 이 한 건이 그 4분의 3을 메웁니다.

	ROI
기본 (상위 10%, 352건)	+65.3%
최고 수익 1건 제외	-8.4%
상위 2건 제외	-10.8%
상위 3건 제외	-13.1%
상위 5건 제외	-16.9%
상위 10건 제외	-25.0%

한 건 빼면 바로 마이너스입니다. 이게 이 보고서에서 가장 중요한 숫자입니다.

9.4 배당 구간별로 쪼개보면

상위 20% 선택(705건) 안에서 배당 구간별 성적입니다.

연승 배당	베팅 수	적중률	평균 배당	ROI
~2배	12	58.3%	1.85	+8.3%
2~3배	142	34.5%	2.67	-8.9%
3~5배	359	17.0%	3.93	-34.3%
5~10배	172	19.8%	6.55	+23.1%
10배 초과	20	15.0%	25.45	+1321.5%

10배 초과 구간의 ROI가 +1321%인데, 20건 중 3건 적중이고 그중 하나가 260배입니다. 통계라기보다 사건에 가깝습니다.

주목할 건 3~5배 구간입니다. 359건으로 가장 많이 베팅했는데 ROI가 -34.3%로 최악입니다. 모델이 이 구간에서는 시장을 못 이깁니다. 실전에서는 이 구간을 아예 제외하는 게 나을 수 있습니다.

9.5 두 모델을 합쳐보면

아이디어는 이렇습니다. 인기마가 무너질 것 같은 경주라면, 그 경주에서는 다크호스가 뚫고 들어올 여지가 큼니다. 그런 경주에서만 베팅하면 어떨까요?

경주별 '붕괴 위험도' = 그 경주 인기마들의 붕괴 확률 중 최대값

위험도 상위 경주만 남기고, 그 안에서 다크호스 상위 20%에 베팅

전략	베팅 수	적중률	ROI	1건 제외 시
다크호스 상위 20% (전 경주)	705	21.8%	+24.0%	-12.8%
+ 붕괴위험 상위 50% 경주만	352	21.6%	+54.1%	-19.6%
+ 붕괴위험 상위 30% 경주만	211	22.3%	+107.1%	-15.9%

ROI가 24% → 54% → 107%로 올라갑니다. 그럴듯해 보입니다.

그런데 마지막 열을 보세요. 260배 적중 1건을 빼면 셋 다 마이너스입니다. 그리고 그 한 건이 세 전략 모두에 들어 있습니다 — 필터를 조일수록 베팅 수가 줄어드니, 같은 수익을 더 적은 베팅으로 나눈 것뿐입니다.

적중률은 21.8% -> 21.6% -> 22.3%로 거의 변하지 않았습니다. 필터가 선별력을 높이지 못했다는 뜻입니다.

ROI만 올라간 건 분모(베팅 수)가 줄어서입니다. 개선이 아니라 착시입니다.

전략을 평가할 때는 ROI보다 적중률과 Lift를 먼저 보세요. ROI는 소수의 고배당에 휘둘립니다.

9.6 그래서 수익이 나는가

정직하게 답하면 — 현재 데이터로는 알 수 없습니다.

- 선별 능력은 확실합니다. 적중률 13.5% → 23.3%, Lift 신뢰구간 [1.41, 2.02]. 이걸 우연이 아닙니다.
- 수익성은 불확실합니다. ROI 신뢰구간이 전부 0을 포함하고, 1건 제외하면 부호가 뒤집힙니다.
- 필요한 건 표본입니다. 고배당 적중은 드물어서, 몇 년치 데이터가 더 있어야 평균이 안정됩니다.

다만 방향은 나쁘지 않습니다. 무작위 베팅이 -22.6%인 시장에서 상위 30%만 골라도 +6.2%가 나왔다는 건, 최소한 공제율(약 20%)만큼은 모델이 메웠다는 뜻입니다. 손익분기 근처까지는 왔습니다.

무작위 -22.6% -> 상위 30% +6.2% -> 상위 10% +65.3%

공제율 20%를 넘어선 것까지는 분명. 그 이상은 표본이 더 필요

10. 권장 설정

바로 쓸 수 있게 정리했습니다.

10.1 최종 선택

항목	다크호스	인기마 붕괴
모델	RandomForestClassifier	RandomForestClassifier
n_estimators	600	600
max_depth	12 (4~12 무방)	8
min_samples_leaf	50 (가장 중요)	50
class_weight	None (쓰지 말 것)	None
학습 범위	pop_pct >= 0.50	pop_pct <= 0.25
타겟	upset_B (입상)	upset_A
주 지표	Lift@10%	Lift@10%
도달 성능	AUC 0.642 / Lift 1.72	AUC 0.577 / Lift 1.47

10.2 코드

```
from sklearn.ensemble import RandomForestClassifier

# 1) 부분집합 추출 — 이게 핵심입니다
tr = train[train.pop_pct >= 0.50].reset_index(drop=True)
te = test[test.pop_pct >= 0.50].reset_index(drop=True)

# 2) 누출 열 제거
DROP = ['pop_pct', 'upset_A', 'upset_B',          # 타겟 정의
        'winOdds', 'plcOdds', 'winAmt', 'plcAmt',  # 시장
        'totalAmt', 'log_winAmt', 'liq_per_horse',
        'ord', 'place', 'win', 'fold',           # 결과
        'race_id', 'entry_id', 'hrName', 'jkName'] # 식별자
X_tr = tr.drop(columns=DROP)

# 3) 학습
m = RandomForestClassifier(
    n_estimators=600,
    max_depth=12,
    min_samples_leaf=50, # 이 값이 성능을 좌우합니다
    random_state=42,
    n_jobs=-1)          # class_weight 지정하지 않음
m.fit(X_tr, tr['upset_B'])
```

4) 예측 — 확률값의 순위만 씁니다

```
p = m.predict_proba(te.drop(columns=DROP))[:, 1]
top10 = te.iloc[p.argsort()[::-1][:int(len(te) * 0.10)]]
```

10.3 체크리스트

- 부분집합으로 먼저 자르고 학습했는가 (전체로 하면 무너집니다)
- pop_pct와 배당률 계열을 전부 뺐는가 (남으면 AUC 0.98이 나옵니다)
- min_samples_leaf를 50 근처로 뒀는가 (기본값 1이면 과적합)
- class_weight를 건드리지 않았는가
- 시간순으로 분할했는가 (무작위 분할은 미래 정보 누출)
- Lift로 평가했는가 (AUC만 보면 잘못 판단합니다)
- 부트스트랩 신뢰구간을 붙였는가
- 수익률은 연승(plcOdds) 배당으로 계산했는가 (단승 쓰면 3배 부풀려집니다)
- 고배당 적중 몇 건을 빼도 결론이 유지되는지 확인했는가

11. 한계

이 결과를 어디까지 믿을 수 있는지 정리합니다.

- **표본이 작습니다** — test 3,528행에 양성 478건. 상위 10%면 352건 중 적중 82건입니다. 이 정도면 지표 하나가 0.02쯤 흔들리는 건 정상입니다. 모델 간 순위도 시드에 따라 바뀔 수 있습니다 — 특히 인기마 붕괴 쪽은 RF(0.5768)와 XGBoost(0.5783) 차이가 무의미한 수준입니다.
- **수익성이 검증되지 않았습니다** — 9장에서 본 대로 ROI는 260배 적중 1건에 전적으로 의존합니다. 실전 투입 판단의 근거로 쓸 수 없습니다. 최소 3~5년치 데이터로 다시 확인해야 합니다.
- **서울 경마장만 봤습니다** — 부경(부산경남)에서도 같은 결과가 나오는지 확인하지 않았습니다. 트랙 특성과 참가마 수준이 달라서 다른 결과가 나올 수 있습니다. 이견 외부 검증 기회이기도 합니다.
- **최근 20%가 test입니다** — 시장 효율성이 해마다 조금씩 올라가고 있습니다(2023년 0.8031 → 2025년 0.8193). test 구간이 가장 효율적인 시기라, 성능이 과소평가됐을 수도 있고 반대로 최근 패턴에만 맞은 것일 수도 있습니다.
- **대안을 다 보지 못했습니다** — ExtraTrees, CatBoost, 신경망 계열은 시도하지 않았습니다. 다만 랜덤포레스트가 부스팅을 이긴 이유(표본 부족·약한 신호)가 구조적이라, 더 복잡한 모델이 이길 가능성은 낮다고 봅니다.
- **타겟 정의는 데이터에서 역산한 것입니다** — $\text{upset_A} \cdot \text{upset_B}$ 의 원래 설계 의도를 담당자에게 확인하지 못했습니다. 일치율 100%로 재현했으니 계산식은 맞지만, 왜 25%와 50%를 임계로 잡았는지는 모릅니다.

12. 다음에 할 일

	할 일	왜	난이도
1	부경 데이터로 같은 실험	외부 검증. 재현되면 결과의 신뢰도가 크게 올라감	낮음
2	3~5년치로 ROI 재계산	고배당 적중이 충분히 쌓여야 평균이 안정됨	낮음
3	배당 구간별 전략 분리	3~5배 구간 ROI -34.3%. 이 구간 제외 여부 검토	낮음
4	임계값 민감도 확인	인기 50%/25% 대신 40%·60% 등으로 바뀌도 결론이 유지되는지	중간
5	경주 유형별 세분화	거리·등급·트랙 상태별로 모델을 나누면 나아지는지	중간
6	시장 효율성 지도와 대조	이변 빈도와 시장 효율성의 상관관계 -0.968. 같은 현상의 두 얼굴	중간
7	Benter식 시장 결합	모델 확률과 시장 확률을 가중 결합. 조건부 로짓에서 델타 R2 +0.0048 확인됨	높음

1~3번은 코드를 거의 그대로 재사용할 수 있어서 하루면 됩니다. 특히 2번은 지금 보고서에서 가장 약한 부분(수익성)을 직접 겨냥합니다.

부록 A. 용어

용어	뜻
기저율	아무 정보 없이 무작위로 골랐을 때의 적중률. 모든 지표의 기준점
ROC-AUC	양성과 음성을 하나씩 뽑았을 때 모델이 양성에 더 높은 점수를 줄 확률. 0.5가 무작위
PR-AUC	정밀도-재현율 곡선 아래 면적. 양성이 드물 때 ROC-AUC보다 민감. 기저율이 무작위 수준
Lift@k%	상위 k%의 적중률을 기저율로 나눈 값. 1.0이면 무작위와 같음
배깅	데이터를 여러 번 복원추출해 모델을 따로 만들고 평균내는 방식. 랜덤포레스트가 이것
부스팅	앞선 모델이 틀린 부분에 집중해 다음 모델을 만드는 방식. XGBoost·LightGBM이 이것
min_samples_leaf	결정나무 잎에 최소 몇 개의 데이터가 있어야 하는지. 클수록 과적합이 줄어들
부트스트랩	데이터를 복원추출로 다시 뽑아 여러 번 계산해 신뢰구간을 만드는 방법
단승 / 연승	단승은 1착만 적중. 연승은 3착까지 적중. 연승이 적중률은 높고 배당은 낮음
파리뮤추얼	판매액에서 주최측 몫을 떼고 남은 돈을 적중자끼리 나누는 방식. 배당이 사후에 결정됨
공제율	주최측이 떼는 비율. 한국마사회 단승 기준 약 20%. 무작위 베팅의 기대 손실이 이만큼

부록 B. 실험 수치 전문

B.1 다크호스 (test 3,528 / 기저율 13.55%)

모델	ROC-AUC	PR-AUC	L@5%	L@10%	L@20%	L@30%
무작위	0.4881	0.1344	1.09	1.07	0.95	0.96
로지스틱 회귀	0.6195	0.1928	1.72	1.59	1.55	1.49
랜덤포레스트 (d12)	0.6420	0.2093	2.01	1.72	1.61	1.45
XGBoost (d3)	0.6205	0.2058	1.89	1.70	1.49	1.43
LightGBM	0.6034	0.1803	1.34	1.38	1.35	1.33

RF Lift@10% 부트스트랩 95% CI = [1.41, 2.02] (2,000회)

피쳐 중요도 상위 10

train_runs_14_z, hr_last_ord, hr_last_poppct, hr_rest_days_z,
rating_z, hr_plcrate, tr_winrate_z, wg_diff_z, wgBudam_z, hr_style_sd

B.2 인기마 붕괴 (test 1,902 / 기저율 25.45%)

모델	ROC-AUC	PR-AUC	L@5%	L@10%	L@20%	L@30%
무작위	0.4870	0.2495	0.95	0.85	0.95	0.92
로지스틱 회귀	0.5722	0.3134	1.53	1.47	1.34	1.21
랜덤포레스트 (d8)	0.5768	0.3177	1.49	1.47	1.28	1.23
XGBoost (d3)	0.5783	0.3238	1.45	1.41	1.31	1.29
LightGBM	0.5610	0.3097	1.57	1.39	1.26	1.18

RF Lift@10% 부트스트랩 95% CI = [1.21, 1.75] (2,000회)

피쳐 중요도 상위 10

age_pr, hr_rest_days_z, rating_z, train_runs_14_z, wg_diff_z,
jk_winrate, hr_rest_days, jk_resid, hrNo, jk_plcrate

B.3 불균형 처리 / 학습 범위

불균형 처리 (test ROC-AUC, 다크호스)

랜덤포레스트	0.6381	->	0.6264	(-0.0117)
로지스틱 회귀	0.6195	->	0.6196	(+0.0001)
XGBoost	0.5293	->	0.4945	(-0.0348)

학습 범위 (test ROC-AUC, 다크호스)

부분집합 + 다크호스 라벨	0.6419	<-	채택
전체 + 1차 라벨	0.6303		
전체 + 다크호스 라벨	0.4599	<-	붕괴

B.4 수익성

상위 k% 베팅 (연승 배당, 균등 1,000원)

범위	베팅	적중	적중률	평균배당	ROI	95% CI
상위 2%	70	22	31.4%	7.18	+364.1%	[-32.6%, +1154%]
상위 5%	176	48	27.3%	5.39	+144.0%	[-21.9%, +453%]
상위 10%	352	82	23.3%	4.97	+65.3%	[-22.6%, +227%]
상위 20%	705	154	21.8%	4.89	+24.0%	[-23.8%, +104%]
상위 30%	1058	208	19.7%	5.20	+6.2%	[-26.7%, +62%]
전체균등	3528	478	13.5%	8.04	-8.2%	[-30.3%, +20%]

상위 10% 안정성 — 고배당 적중 제외

기본 +65.3% / -1건 -8.4% / -3건 -13.1% / -5건 -16.9% / -10건 -25.0%

적중마 최고 배당: 260.3배 (2위는 9.4배)

배당 구간별 (상위 20% 선택 705건 내)

~2배	n=12	적중 58.3%	ROI +8.3%
2~3배	n=142	적중 34.5%	ROI -8.9%
3~5배	n=359	적중 17.0%	ROI -34.3%
5~10배	n=172	적중 19.8%	ROI +23.1%
10배+	n=20	적중 15.0%	ROI +1321.5%

조합 전략 (붕괴위험 필터 + 다크호스 상위 20%)

전 경주	n=705	적중 21.8%	ROI +24.0%	(1건 제외 -12.8%)
위험 상위 50%	n=352	적중 21.6%	ROI +54.1%	(1건 제외 -19.6%)
위험 상위 30%	n=211	적중 22.3%	ROI +107.1%	(1건 제외 -15.9%)

부록 C. 이 보고서를 세 문장으로

핵심 세 가지

1. 랜덤포레스트를 쓰고, min_samples_leaf를 50으로 두고, 불균형 처리는 하지 마세요. 부분집합에서만 학습하는 게 가장 중요합니다.
2. 평가는 ROC-AUC가 아니라 Lift@10%로 하세요. 다크호스 1.72배 [1.41, 2.02], 인기마 붕괴 1.47배 [1.21, 1.75] — 둘 다 신호가 있습니다.
3. 수익성은 아직 모릅니다. 표면 ROI +65%는 260배 적중 한 건이 만든 숫자이고, 빼면 -8.4%입니다. 몇 년치를 더 봐야 답할 수 있습니다.